Semaine 7 — Première fonctionnalité avec GitHub Copilot

Objectif de la séance

Implémenter une fonctionnalité complète, du frontend à la base de données, avec GitHub Copilot. Vous devez pouvoir expliquer les modifications et montrer comment vous avez vérifié le résultat.

Fonctionnalité demandée : les candidatures

À partir du cahier des charges, implémentez :

- **EF-32** : un étudiant candidate à une offre publiée, avec un message facultatif. Il peut retirer sa candidature tant qu'elle n'a pas été traitée.
- **EF-33** : l'étudiant consulte ses candidatures et leur statut : « En cours », « Retenue », « Non retenue » ou « Retirée ».

Utilisez la liste des offres et les connexions de démonstration existantes. Le bouton « Candidater » peut être placé dans cette liste. EF-30, EF-31, la sélection d'un candidat et la création du stage restent hors périmètre.

1. Préparer avec le mode Plan

Dans le chat GitHub Copilot de VS Code, sélectionnez **Plan** dans le sélecteur d'agents. Selon votre version, la planification peut aussi être accessible avec `/plan`. Ce mode analyse le projet et propose une démarche avant l'implémentation.

Donnez à Copilot le cahier des charges, les exigences concernées et les limites de la séance. Demandez-lui :

- d'examiner le code et le schéma existants ;
- de proposer trois étapes : candidater, consulter ses candidatures, retirer une candidature ;
- d'indiquer les fichiers concernés, les vérifications et les tests envisagés ;

— de signaler les décisions métier à clarifier.

Relisez le plan avant de lancer l'implémentation. Clarifiez notamment les candidatures en double et la possibilité de candidater à nouveau après un retrait. Le cahier des charges ne tranche pas ces points : choisissez un comportement et notez-le.

2. Implémenter progressivement

Lancez l'implémentation du plan avec le mode **Agent**, une étape à la fois. Après chaque étape :

— consultez les modifications dans l'onglet de contrôle de code source de VS Code ;
— identifiez les fichiers modifiés et leur rôle ;
— essayez le comportement obtenu avant de poursuivre ;
— demandez une correction précise si nécessaire.

Gardez des demandes ciblées et réutilisez les éléments existants. Faites des commits compréhensibles pour garder une trace du travail.

3. Vérifier le résultat

Validation fonctionnelle

Avant les essais, notez quelques cas et le résultat attendu. Vérifiez vous-mêmes dans le navigateur, notamment :

— candidature à une offre publiée, avec et sans message ;
— présence de la candidature et de son statut après rechargement ;
— retrait d'une candidature encore en cours ;
— impossibilité de retirer une candidature déjà traitée ;
— séparation des candidatures de deux étudiants différents.

Vérifiez aussi les refus : un autre rôle ne peut pas candidater et une offre non publiée n'accepte pas de candidature. Les contrôles doivent exister côté serveur. Pour un cas inaccessible dans l'interface, demandez à Copilot comment le vérifier par l'API ou un test ciblé.

Validation de la qualité

— **Respect de l'existant** : architecture et conventions de codage.
— **Lisibilité et simplicité** : noms explicites, absence de code inutile ou de complexité injustifiée, capacité à expliquer le code ajouté.
— **Vérifications techniques** : lancez `npm run check` et consultez les résultats du lint, de TypeScript, des tests et de la compilation.

Repérez les tests ajoutés et ceux réellement exécutés. Expliquez ce que vérifie un test ajouté ; si aucun n'a été ajouté, signalez-le. Vérifiez les résultats vous-mêmes, au-delà du compte rendu de Copilot.

4. Livrable : un court bilan

Dans le dossier **`docs/`** du dépôt, ajoutez votre court bilan dans **`semaine-2.md`**.

Quelques paragraphes ou des listes suffisent. Le bilan doit contenir :

- **choix** : quels sont les choix que vous avez pris lors de la plannification
- **Résultat** : ce qui fonctionne, ce qui n'a pas fonctionné, les difficultés et les limites restantes.
- **Modifications** : les fichiers ajoutés ou modifiés et pourquoi ; les décisions prises après lecture du plan.
- **Validation fonctionnelle** : les cas réellement essayés, les résultats observés et les corrections éventuelles.
- **Qualité** : les points vérifiés dans le code, les problèmes repérés et les simplifications effectuées, s'il y en a.
- **Tests et commandes** : les tests ajoutés, les commandes exécutées et leurs résultats ; un exemple de comportement couvert par un test.

Remettez le code et le bilan dans votre dépôt Git, avec les commits poussés sur GitHub. Décrivez votre travail réel, y compris les vérifications non réalisées et les problèmes encore présents.

Documentation du mode Plan dans GitHub Copilot · Planification dans VS Code